

P1_1-2_Setup_CF_KNN

August 14, 2026

1 Programming Assignment 1: Recommender System

1.0.1 Coure: CS373

Developer 1: Jalin A. Brown

Developer 2: Sandeep Durga

File: P1_1-2_Setup_CF_KNN.ipynb

1.0.2 Part One: Data Setup and User-Based Collaborative Filtering

1. Load and preprocess data:

- Create user-item rating matrix.
- Calculate percentage of missing ratings: $\text{Sparsity} = (\text{Number of Missing Ratings} / \text{Total Possible Ratings}) \times 100$

2. Build K-Nearest Neighbors recommender:

- Find k most similar users for any target user (test different values of k)
- Only consider users with positive similarity scores
- Predict ratings using weighted average
- Handle edge case

```
[1]: import numpy as np                                # math
import pandas as pd                                    # user-item matrix
from sklearn.metrics.pairwise import cosine_similarity # pairwise similarity
```

1.0.3 1. Load and preprocess data:

```
[2]: # Create user-item rating matrix
user_item_base = pd.read_csv("../data/u1.base",
    ↪names=["user_id", "item_id", "rating", "timestamp"], sep="\t", )
user_item_test = pd.read_csv("../data/u1.test",
    ↪names=["user_id", "item_id", "rating", "timestamp"], sep="\t")

# Display info
print(f"\nTraining Data Matrix: {user_item_base.shape}\n")
print(user_item_base.head())
```

```
print(f"\nTest Data Matrix: {user_item_test.shape}\n")
print(user_item_test.head())
```

Training Data Matrix: (80000, 4)

	user_id	item_id	rating	timestamp
0	1	1	5	874965758
1	1	2	3	876893171
2	1	3	4	878542960
3	1	4	3	876893119
4	1	5	3	889751712

Test Data Matrix: (20000, 4)

	user_id	item_id	rating	timestamp
0	1	6	5	887431973
1	1	10	3	875693118
2	1	12	5	878542960
3	1	14	5	874965706
4	1	17	3	875073198

```
[3]: # Cleaning: Exclude possible duplicates & non int vals

# training user-item matrix
train_users = user_item_base["user_id"].astype(int).unique()
train_items = user_item_base["item_id"].astype(int).unique()
user_item_training_matrix = pd.DataFrame(np.nan, index=train_users,
    ↪columns=train_items)
for row in user_item_base.itertuples(index = False):
    user_item_training_matrix.loc[row.user_id, row.item_id] = row.rating

# test user-item matrix
test_users = user_item_test["user_id"].astype(int).unique()
test_items = user_item_test["item_id"].astype(int).unique()
user_item_test_matrix = pd.DataFrame(np.nan, index=test_users,
    ↪columns=test_items)
for row in user_item_test.itertuples(index = False):
    user_item_training_matrix.loc[row.user_id, row.item_id] = row.rating

# Display info
print(f"\nTraining User-Item Matrix: {user_item_training_matrix.shape}\n")
print(user_item_training_matrix.head())

print(f"\nTest User-Item Matrix: {user_item_test_matrix.shape}\n")
print(user_item_test_matrix.head())
```

Training User-Item Matrix: (943, 1682)

	1	2	3	4	5	7	8	9	11	13	...	1533	\
1	5.0	3.0	4.0	3.0	3.0	4.0	1.0	5.0	2.0	5.0	...	NaN	
2	4.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	...	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	NaN	...	NaN	
5	4.0	3.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	

	1536	1543	1557	1561	1562	1563	1565	1582	1586
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

[5 rows x 1682 columns]

Test User-Item Matrix: (459, 1410)

	6	10	12	14	17	20	23	24	27	31	...	1565	\
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	

	1578	1582	1586	1470	1522	1121	1038	1591	1269
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

[5 rows x 1410 columns]

```
[9]: # Analyze sparsity
user_count = user_item_training_matrix.shape[0]
item_count = user_item_training_matrix.shape[1]

mat_size = user_count * item_count
present = user_item_training_matrix.count().sum()
missing = mat_size - present
sparsity = (missing / mat_size) * 100

print(f"sparsity: {sparsity}\n")
```

sparsity: 93.69533063577546

1.0.4 2. Build KNN Recommender:

This next module covers:

- Full similarity space with Nan set to 0
- Only keep users with cosine_similarity > 0 (positive relationship)

Note: - Positive similarity (0.0, 1.0] indicates similar taste - Negative similarity [-1, 0.0) indicates opposite taste - Each index [i,j] is the cosine similarity score between user i and j

```
[ ]: def cosine_sim(u, v, m):
    a = m.loc[u]
    b = m.loc[v]
    I_uv = a.dropna().index.intersection(b.dropna().index)
    if len(I_uv) == 0:
        return 0.0
    r_u = a.loc[I_uv].to_numpy()
    r_v = b.loc[I_uv].to_numpy()

    num = np.dot(r_u, r_v)
    den = np.linalg.norm(r_u) * np.linalg.norm(r_v)
    return 0.0 if den == 0 else num / den

[5]: # Fill missing ratings with 0 for cosine similarity
ui_training_mat_filled = user_item_training_matrix.fillna(0)

# Compute cosine similarity between all users
cosine_sim_mat = cosine_similarity(ui_training_mat_filled)
cosine_sim_matrix = pd.DataFrame(cosine_sim_mat,
    ↪ index=user_item_training_matrix.index, columns=user_item_training_matrix.
    ↪ index)

cosine_sim_matrix.head()
```

```
[5]:
```

	1	2	3	4	5	6	7	\
1	1.000000	0.166931	0.047460	0.064358	0.378475	0.430239	0.440367	
2	0.166931	1.000000	0.110591	0.178121	0.072979	0.245843	0.107328	
3	0.047460	0.110591	1.000000	0.344151	0.021245	0.072415	0.066137	
4	0.064358	0.178121	0.344151	1.000000	0.031804	0.068044	0.091230	
5	0.378475	0.072979	0.021245	0.031804	1.000000	0.237286	0.373600	

	8	9	10	...	934	935	936	937	\
1	0.319072	0.078138	0.376544	...	0.369527	0.119482	0.274876	0.189705	
2	0.103344	0.161048	0.159862	...	0.156986	0.307942	0.358789	0.424046	
3	0.083060	0.061040	0.065151	...	0.031875	0.042753	0.163829	0.069038	
4	0.188060	0.101284	0.060859	...	0.052107	0.036784	0.133115	0.193471	
5	0.248930	0.056847	0.201427	...	0.338794	0.080580	0.094924	0.079779	

	938	939	940	941	942	943
1	0.197326	0.118095	0.314072	0.148617	0.179508	0.398175
2	0.319889	0.228583	0.226790	0.161485	0.172268	0.105798
3	0.124245	0.026271	0.161890	0.101243	0.133416	0.026556
4	0.146058	0.030138	0.196858	0.152041	0.170086	0.058752
5	0.148607	0.071459	0.239955	0.139595	0.152497	0.313941

[5 rows x 943 columns]

Find K most similar users for any target user (positive similarity only, testing different values of K)

using user 1 as an example

```
[6]: # Select the target user
current_user = 1

# Select a value of K
K_values = [1, 3, 5, 7, 10]

# Extract similarities between this user and all other users
similarities_to_others = cosine_sim_matrix[current_user].copy()
similarities_to_others = similarities_to_others.drop(index=current_user)
pos_sim = similarities_to_others[similarities_to_others > 0.0].
    ↪sort_values(ascending=False)

print(f"Top similar users for user {current_user} with varying K:\n")
for k in K_values:
    top_k = pos_sim.head(k)
    print(f"K = {k}:")
    print(top_k)
    print("-"*40)
```

Top similar users for user 1 with varying K:

K = 1:

916 0.569066

Name: 1, dtype: float64

K = 3:

916 0.569066

864 0.547548

268 0.542077

Name: 1, dtype: float64

K = 5:

916 0.569066

```

864    0.547548
268    0.542077
92     0.540534
435    0.538665
Name: 1, dtype: float64
-----
K = 7:
916    0.569066
864    0.547548
268    0.542077
92     0.540534
435    0.538665
457    0.538476
738    0.527031
Name: 1, dtype: float64
-----
K = 10:
916    0.569066
864    0.547548
268    0.542077
92     0.540534
435    0.538665
457    0.538476
738    0.527031
429    0.525950
303    0.525718
276    0.524523
Name: 1, dtype: float64
-----

```

1.0.5 Predict ratings using weighted average

Using the weighted similarity formula: - Select target user (u) and a target item (i) for which the user has not rated yet - Gather the K most similar neighbor users who have rated the target item - Compute the weighted average of their ratings using similarity as weight - If no neighbor has rated that item, handle as an edge case

```
[7]: # Check Target user for an unrated target item
user_item_training_matrix
```

```
[7]:
```

	1	2	3	4	5	7	8	9	11	13	...	1533	\
1	5.0	3.0	4.0	3.0	3.0	4.0	1.0	5.0	2.0	5.0	...	NaN	
2	4.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	...	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.0	NaN	...	NaN	
5	4.0	3.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	
..	...	...	...	...	...	...	...	...	...	...	...	...	
939	NaN	NaN	NaN	NaN	NaN	NaN	NaN	5.0	NaN	NaN	...	NaN	

940	NaN	NaN	NaN	2.0	NaN	4.0	5.0	3.0	NaN	NaN	...	NaN
941	5.0	NaN	NaN	NaN	NaN	4.0	NaN	NaN	NaN	NaN	...	NaN
942	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN
943	NaN	5.0	NaN	NaN	NaN	NaN	NaN	3.0	4.0	NaN	...	NaN

	1536	1543	1557	1561	1562	1563	1565	1582	1586
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
..	...	...	...	...	...	...	...	...	...
939	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
940	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
941	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
942	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
943	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

[943 rows x 1682 columns]

```
[8]: # Choose target user and unrated target item
target_user = 2
target_item = 2
k_value = 5

# Gather k most similar neighbors who have rated the target item
pos_sim = cosine_sim_matrix[current_user].drop(index=current_user)
pos_sim = pos_sim[pos_sim > 0.0].sort_values(ascending=False)
top_k = pos_sim.head(k_value)

numerator = 0.0
denominator = 0.0

for neighbor, sim in top_k.items():
    neighbor_rating = user_item_training_matrix.loc[neighbor, target_item]
    if not np.isnan(neighbor_rating):
        numerator += neighbor_rating * sim
        denominator += abs(sim)
if denominator == 0:
    print(f"No similar users for user {target_user} rated {target_item}.")
    predicted_rating = np.nan
else:
    predicted_rating = numerator / denominator
    print(f"Using K value {k_value}, the predicted rating for user_
    ↪{target_user} on item {target_item} is {predicted_rating:.2f}.")
```

Using K value 5, the predicted rating for user 2 on item 2 is 3.20.